CERONIX SUPPLY CHAIN RISK ANALYSIS SYSTEM

Comprehensive Architecture Documentation

Generated on: September 27, 2025 at 11:18 AM

TABLE OF CONTENTS

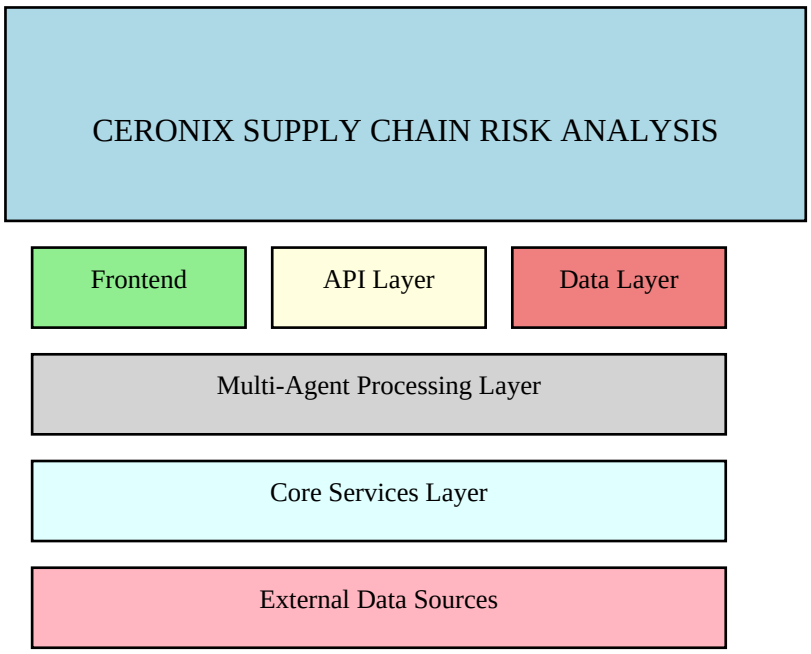
1. SYSTEM OVERVIEW

The CERONIX system is a comprehensive, multi-layered architecture that combines data collection agents, real-time processing, web services, and machine learning capabilities for supply chain risk analysis. The system provides end-to-end visibility into supply chain risks through automated data collection, intelligent processing, and interactive visualization.

Key Features:

- Multi-Agent Architecture with 8 specialized agents
- Real-time data collection from multiple sources
- Google Maps integration for geocoding and location services
- Interactive React-based web dashboard
- FastAPI backend with comprehensive REST endpoints
- MongoDB database for scalable data storage
- Docker containerization for deployment
- Prefect-based workflow orchestration

2. HIGH-LEVEL SYSTEM ARCHITECTURE



The system follows a layered architecture pattern with clear separation of concerns:

Frontend Layer (React): Interactive web dashboard with real-time updates, responsive design, and modern UI components

API Layer (FastAPI): RESTful API server with WebSocket support, comprehensive endpoints, and authentication

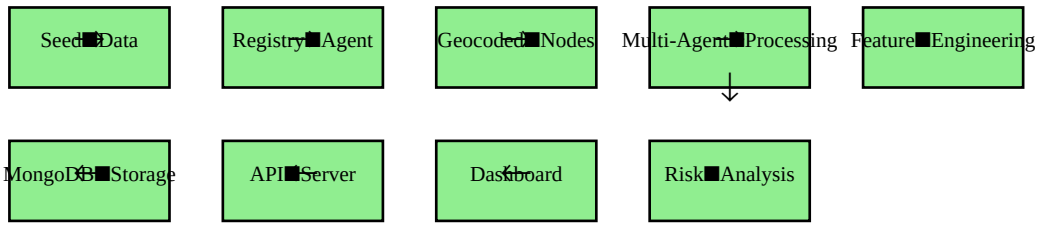
Data Layer (MongoDB): Scalable NoSQL database with collections for nodes, events, features, and analytics

Multi-Agent Processing: 8 specialized agents for data collection, processing, and feature engineering

Core Services: Reusable services for geocoding, NLP, news processing, and utility functions

External Data Sources: Integration with Google Maps, SERP API, Weather API, Twitter API, and RSS feeds

3. DATA FLOW ARCHITECTURE



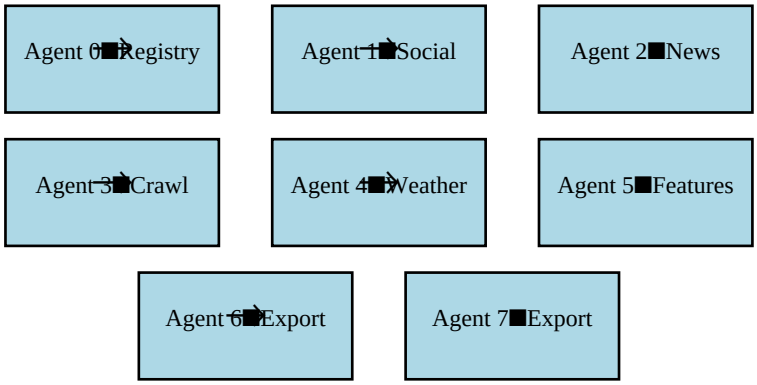
Primary Data Flow:

```
Seed Data → Registry Agent → Geocoded Nodes → Multi-Agent Processing →
Feature Engineering → Risk Analysis → Dashboard
```

Detailed Flow Steps:

1. Data Ingestion: Seed supplier data from CSV files
2. Registry Processing: Agent 0 normalizes and geocodes suppliers using Google Maps
3. Multi-Source Collection: Agents 1-4 collect data from various sources
4. Feature Engineering: Agent 5 processes and combines all data
5. Export & Storage: Agents 6-7 export to CSV and store in MongoDB
6. API Serving: FastAPI serves data to React frontend
7. Real-time Updates: WebSocket provides live updates

4. AGENT ARCHITECTURE DETAILS



Agent 0 - Registry Agent

Purpose: Supplier registry normalization and geocoding

Input: Raw supplier CSV data

Processing: Data cleaning, Google Maps geocoding, duplicate detection

Output: Geocoded supplier nodes

Dependencies: Google Maps API, core.geo services

Agent 1 - Social Media Agent

Purpose: X/Twitter data collection

Input: Supplier names and handles

Processing: Twitter API integration, sentiment analysis, event extraction

Output: Social media events and sentiment scores

Dependencies: Twitter API, NLP services

Agent 2 - News Agent

Purpose: News article collection and processing

Input: Supplier names and regions

Processing: RSS feed monitoring, SERP API searches, content analysis

Output: News events with sentiment and relevance scores

Dependencies: SERP API, RSS feeds, NLP services

Agent 3 - Crawl Agent

Purpose: Deep web crawling and content extraction
Input: URLs from news and social media
Processing: crawl4ai integration, HTML/PDF processing, event extraction
Output: Structured events and extracted data
Dependencies: crawl4ai, rate limiting services

Agent 4 - Weather Agent

Purpose: Weather anomaly detection
Input: Supplier locations (lat/lon)
Processing: Weather API integration, anomaly detection, risk scoring
Output: Weather anomalies and risk scores
Dependencies: Weather API, geographic services

Agent 5 - Features Agent

Purpose: ML feature engineering
Input: All collected data from agents 1-4
Processing: Data aggregation, feature calculation, risk score computation
Output: ML-ready feature vectors
Dependencies: All previous agents, core.utils

Agent 6 - Export Agent

Purpose: CSV export for ML pipeline
Input: Feature vectors from Agent 5
Processing: Data validation, CSV formatting, file output
Output: features_today.csv
Dependencies: Agent 5, core.io services

Agent 7 - Export Agent (Extended)

Purpose: Additional reporting and analytics
Input: All processed data
Processing: Report generation, analytics computation, dashboard data preparation
Output: Reports and analytics data
Dependencies: All agents, MongoDB

5. WEB ARCHITECTURE

Frontend Layer (React):

• App.js - Main application component

• Components - Dashboard, Suppliers, Alerts, Risk Monitoring

• Hooks - useApiData, useWebSocket, useMetrics for data management

• Services - API calls, WebSocket connections, data processing

• Utils - Helper functions, constants, configuration

• Styles - CSS/SCSS with responsive design

Backend API Layer (FastAPI):

• main.py - Routes, middleware, CORS configuration

• services.py - Business logic and data processing

• models.py - Pydantic models for validation

• database.py - MongoDB connection and models

• WebSocket - Real-time updates and notifications

• Authentication - API key management and security

6. DATABASE ARCHITECTURE

MongoDB Collections:

• **nodes**: Suppliers, locations, metadata

• **news_events**: Articles, sentiment, timestamps

• **extracted_events**: Crawled data, structured events

• **weather_anomalies**: Weather data, anomaly flags

• **features**: ML data, vectors, scores

• **alerts**: Warnings, critical events, notifications

7. CORE SERVICES ARCHITECTURE

• **geo.py**: Geocoding, Maps API, location services
• **nlp.py**: Sentiment analysis, event classification
• **news.py**: RSS feeds, SERP API, content processing
• **io.py**: File I/O, caching, storage management
• **utils.py**: Helper functions, mathematical operations, statistics
• **rate.py**: Throttling, retry logic, rate limiting
• **db_models.py**: MongoDB models, queries, database operations
• **schemas.py**: Validation, schemas, data types
• **google_maps_geocoder.py**: Maps API integration, geocoding services

8. ORCHESTRATION ARCHITECTURE

Prefect Workflow Management:

• flow_daily.py - Daily pipeline execution and agent coordination

• cli.py - Command-line interface for testing, setup, and health checks

• Scheduler - Cron jobs, triggers, and automated execution

• Monitoring - Workflow status, error handling, and alerting

9. DEPLOYMENT ARCHITECTURE

Docker Containerization:

- **Backend Container**: FastAPI, Python, agents, core services
- **Frontend Container**: React, Node.js, Nginx for serving
- **MongoDB Container**: Database, storage, persistence

10. API ENDPOINT ARCHITECTURE

• **/api/health**: System health check and status
• **/api/dashboard**: Dashboard data aggregation
• **/api/suppliers**: Supplier management and CRUD operations
• **/api/risk-factors**: Risk analysis data and metrics
• **/api/routes**: Supply chain routes and logistics
• **/api/alerts**: Alert management and notifications
• **/api/metrics**: System metrics and performance data
• **/ws/updates**: WebSocket for real-time updates

11. SECURITY ARCHITECTURE

- **Environment Variables**: API keys, secrets, configuration management
- **API Security**: CORS, rate limiting, input validation
- **Data Encryption**: MongoDB encryption, file security, network protection

12. MONITORING & LOGGING ARCHITECTURE

- **Logging**: Structured JSON logs with timestamps and context
- **Metrics**: Performance metrics, business metrics, system metrics
- **Health Checks**: System status, component health, alerting

13. TECHNICAL SPECIFICATIONS

Technology Stack:

• **Frontend**: React 18, JavaScript ES6+, CSS3, WebSocket
• **Backend**: FastAPI, Python 3.11+, Pydantic, Uvicorn
• **Database**: MongoDB 6.0+, Motor (async driver)
• **Orchestration**: Prefect 2.0+, asyncio
• **APIs**: Google Maps, SERP API, Weather API, Twitter API
• **Deployment**: Docker, Docker Compose, Nginx
• **Monitoring**: Structured logging, health checks

Performance Characteristics:

• Concurrency: Async/await throughout the system
• Rate Limiting: Configurable per API endpoint
• Caching: In-memory and file-based caching strategies
• Scalability: Horizontal scaling ready architecture
• Real-time: WebSocket for live updates and notifications